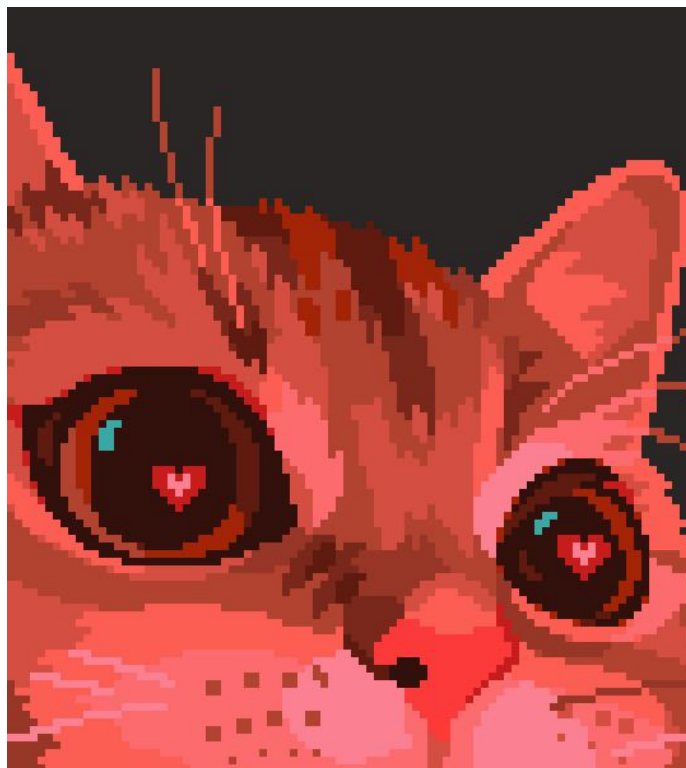




Password Store Audit Report

Aayush Suthar

May 26, 2026

Prepared by: Aayush Suthar

Table of Contents

- Table of Contents
- Disclaimer
- Risk Classification
- Protocol Summary
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Storing the password on-chain make it visible to anyone, and no longer private
 - * [H-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Disclaimer

The auditor makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Protocol Summary

A smart contract application for storing a password. Users should be able to store a password and then retrieve it later. Others should not be able to access the password.

Audit Details

The findings described in this document corresponds the following commit hash:

2e8f81e263b3a9d18fab4fb5c46805ffc10a9990

Scope

```
./src/  
    PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsides: No one else should be able to set or read the password.

Executive Summary

Executive Summary

This audit was conducted as a guided learning exercise through the Cyfrin Updraft Smart Contract Security course. The PasswordStore protocol was reviewed as part of the course curriculum to develop practical auditing skills and familiarity with the audit process end to end.

The review was conducted by Aayush Suthar following the methodology taught in the course, including manual code review, static analysis, and proof of concept development using Foundry.

Despite being a learning exercise, the findings documented in this report represent real vulnerabilities present in the codebase. Two High severity issues and one Informational finding were identified during the review.

Tools used: Foundry, Manual Review

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] Storing the password on-chain make it visible to anyone, and no longer private

Description: All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract.

We show one such method of reading any data off chain directly

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: (Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool

We use 1 because that's the storage slot of `s_password` in the contract.

```
cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this:

```
0x6d7950617373776f726440000000000000000000000000000000000000000014
```

You can then parse that hex to a string with:

```
cast parse-bytes32-string 0x6d7950617373776f72640000000000000000000000000000000000000000000014
```

```

    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }

```

Informational

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Description:

```

    /*
     * @notice This allows only the owner to retrieve the password.
    @> * @param newPassword The new password to set.
     */
    function getPassword() external view returns (string memory) {

```

The natspec for the function `PasswordStore::getPassword` indicates it should have a parameter with the signature `getPassword(string)`. However, the actual function signature is `getPassword()`.

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```

-     * @param newPassword The new password to set.

```